

qf-pgpe: a verified pure-Rust projected Gross–Pitaevskii engine for quantum-fluid experiments, with a Python extension

The quantum-fluids module of rusty-SUNDIALS — design, cross-language verification, performance, and what porting found

Xavier Callens

SocrateAI-Scientific-QuantumFluids and rusty-SUNDIALS

<https://github.com/xaviercallens/rusty-SUNDIALS>

October 2026

Abstract

Classical-field (projected Gross–Pitaevskii) simulations of two-dimensional Bose fluids are usually scripts: an FFT library, a time stepper and analysis code whose correctness is checked, when at all, by a few conservation laws. We describe **qf-pgpe**, the quantum-fluids module of the rusty-SUNDIALS numerical library: a pure-Rust integrating-factor RK4 engine for the projected equation on a periodic grid, a vortex instrument (periodic theta-function imprint, sub-grid detection, tracking), the registered estimators of vortex friction and diffusion, a thermal-state toolkit and a vortex–phonon scattering instrument, plus a Python extension that exposes them on numpy arrays. Every module is a port of a Python original of an ongoing, pre-registered research programme and carries a numeric cross-check against it: estimators agree to 2×10^{-14} on real vortex tracks, a 31-run scattering matrix to 5×10^{-11} , measurements on a shared thermal field to 10^{-14} ; the engine is checked against independent known answers and against rusty-SUNDIALS’ own CVODE (global error falling as $\Delta t^{4.00}$). The cross-checks found three defects, one of them in the Python programme: its primary estimator of the mutual friction is biased low by vortex diffusion (-8 to -22%), a flaw the registered synthetic gate had not shown. On a single thread of a 2013 laptop CPU the engine is $4.3\times$, $4.2\times$, $2.7\times$ and $2.5\times$ faster per step than the numpy engine at $N = 64, 128, 256, 512$ ($3.2\times$ – $1.5\times$ against the fastest of the Python engines we tried, torch on CPU), and a drop-in replacement of the Python stepper makes an end-to-end thermal-state measurement $5.3\times$ faster with identical results. We make no claim about GPU codes or other published solvers, which we did not measure; intra-run threading, tried, did not help on this machine.

1 Introduction

Finite-temperature quantum-fluid physics in two dimensions — the Berezinskii–Kosterlitz–Thouless transition, the friction on a quantized vortex, the screening by thermal vortex pairs — is routinely studied with the projected Gross–Pitaevskii equation (PGPE) [1, 2]: a microcanonical classical field $i\partial_t\psi = P[-\frac{1}{2}\nabla^2\psi + g|\psi|^2\psi]$ with a sharp momentum cutoff P that makes the thermal cloud, the condensate and the vortices dynamical. The numerical core is small (a 2D FFT pair and a Runge–Kutta stepper); what is large, and error-prone, is what sits on top of it: imprinting vortices on a torus, finding and following them, reading transport coefficients out of the tracks, measuring temperature and normal density of a state.

This paper describes the module of the rusty-SUNDIALS library [3] (a Rust implementation of the SUNDIALS solver suite) that carries this stack, **qf-pgpe**, and records what we can and cannot say about it. Its design rule is that no module is accepted without a direct numeric comparison against the original it replaces, on the same inputs. Our contributions are: (i) the engine and

instruments (Section 2); (ii) a verification record that includes independent integration by the library’s own CVODE and cross-language comparisons with quantified differences (Section 3); (iii) the three defects this found (Section 4); (iv) measured performance against numpy, scipy and torch, with the conditions stated (Section 5); (v) a Python extension with a drop-in stepper (Section 6). The physics results that the programme obtained with these tools are published separately [4, 5, 6]; here they serve as use cases (Section 7).

What is claimed, and what is not. Claimed: agreement with the Python originals at the stated levels; single-thread speed-ups over the numpy, scipy and torch-CPU engines on the stated machine; bit-identical results between optimisations. **Not claimed:** that this is the best PGPE solver. We did not benchmark GPU codes, FFTW-based C codes or other published solvers (none were installed), the machine is a 2013 quad-core laptop CPU, and the one parallelisation we tried inside a run gave no speed-up.

2 Design

Engine. `ComplexField2D` integrates the projected equation on an $N \times N$ periodic grid of side L ($\hbar = m = 1$, interaction g) by the integrating-factor RK4 scheme: the linear part is exact in Fourier space, the cubic part is classical RK4, the projector is a sharp circle $|k| \leq k_{\text{cut}} = f k_{\text{max}}$ (default $f = 1/2$, the cutoff at which the cubic term is alias-free in the interior of the retained set). FFT conventions are numpy’s (`fftfreq` ordering; unnormalised forward transform), so arrays are exchanged with the Python programme without conversion. Norm, energy and momentum are provided; `rhs/pack/unpack` expose the right-hand side as an ODE system for external integrators.

Instruments. Table 1 lists the modules. All are ports of the Python programme’s scripts, with the same formulas and the same registered conventions.

module	content	Python original
<code>lib</code>	IF-RK4 PGPE, cutoff fraction, invariants, <code>rhs/pack/unpack</code>	<code>pgpe.py</code>
<code>vortex</code>	periodic theta-function imprint with Bernoulli amplitude, raw plaquette detection with sub-grid refinement, tracker, tracked runs, placements	<code>vortex_transport.py</code>
<code>transport</code>	estimators of α (energy, regression), $1 - \alpha'$, η (residual MSD), block jackknife; point-vortex velocity/energy on the torus; synthetic Langevin gate G0	<code>transport_estimators.py</code>
<code>thermal</code>	seeded RNG, <code>random_state</code> , <code>heat</code> , thermometer, condensate fraction, current correlators (n_s/n), vortex count, Fourier resampling, <code>run_blocks</code> , <code>make_base</code>	<code>observables.py</code> , <code>round2.py</code>
<code>scattering</code>	Bogoliubov-wave dressing of a vortex pair, wave momentum flux, drift slopes, $\sigma_{\parallel}(k)$, $\sigma_{\perp}(k)$; parallel resumable scan runner	<code>vortex_wave_scattering.py</code>
<code>qf-pgpe-py</code>	Python extension (pyo3, numpy)	—

Table 1: The modules of the quantum-fluids crate. Not ported (declared): the g_1 correlation fits, vortex–dipole matching, the Onsager order parameter, persistent-homology instruments and the symbolic series of the programme.

Performance engineering. The step is allocation-free (a workspace is allocated once per run, two ping-pong buffers carry the state) and the 2D FFT is rows–transpose–rows with a tiled

in-place transpose (unit stride in both passes). In the nonlinear term the FFT lines that are identically zero in a projected field (rows with no mode inside the projector on the way in) or are discarded by the projector (on the way out) are not transformed; this is exact (a test asserts equality bit for bit for three cutoffs) and gave 8–11 %. A rayon-parallel pass over rows was implemented and removed: on this four-core machine it gave no speed-up at $N = 512$ and 1024 with two and four threads, and independent runs — the unit in which the programme parallelises — scale across processes (Section 5).

3 Verification

Known answers. The engine carries the programme’s pre-registered known answers: norm conservation (10^{-8}), momentum conservation (10^{-10} for states with weight away from the cutoff edge), the exact plane-wave solution (10^{-9}), and, for the vortex instrument, the $T = 0$ gate of the campaign (a neutral pair neither shrinks nor changes momentum; momentum 0.96–0.99 of $2\pi nd$; exactly two detections).

An independent integrator: CVODE. The right-hand side of the projected equation, on the retained modes, is an ODE system of dimension $2N_{\text{modes}}$. We hand it, unchanged, to rusty-SUNDIALS’ variable-order Adams solver with tight tolerances ($N = 16$, $L = 16$, 49 modes, $t = 1$) and use the result as an independent reference. The global error of the IF-RK4 step falls as $\Delta t^{4.00}$ over $\Delta t = 0.04 \dots 0.0025$ (Fig. 1, left), which is the order the programme registered (its “K2” check, previously not ported). The test depends on a fix of the library (the Adams order was stuck at one): without it the solver collapses its step on this oscillatory Hamiltonian problem and fails after about 10^6 right-hand-side evaluations; with it, 482 evaluations in 95 steps. The test is therefore also a regression guard for the solver.

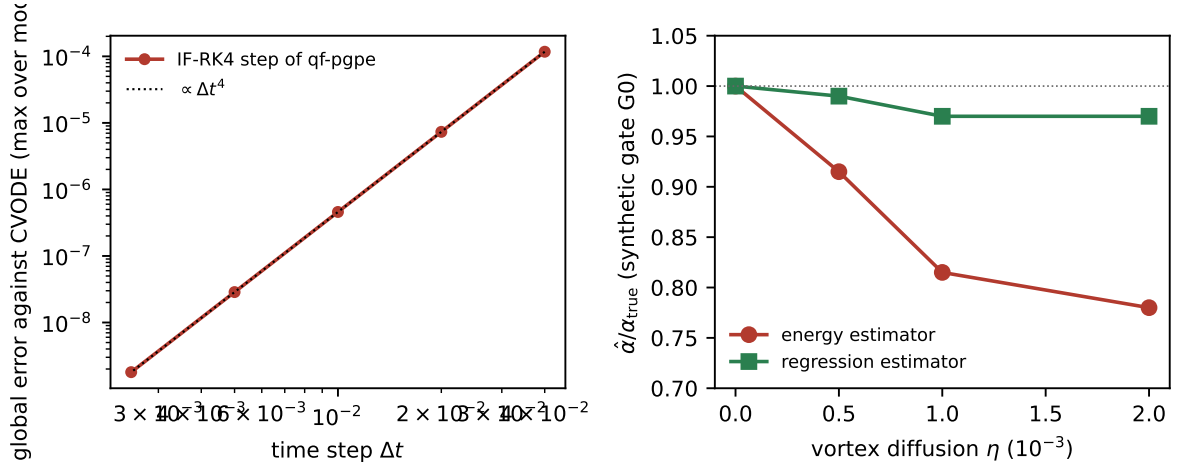


Figure 1: Left: global error of the IF-RK4 step against a CVODE reference of the same right-hand side. Right: the energy estimator of the friction is biased low by diffusion in the registered synthetic gate (a flaw found by the port, Section 4).

Cross-language agreement. Table 2 gives the measured differences between each Rust module and its Python original on the same inputs.

A statistical comparison that did not meet its criterion. For the thermal-state constructor ($N = 64$, $L = 32$, $e = 0.6$, nine seeds each) the temperatures are 0.0757 (Rust) and 0.0865 (Python), per-seed standard deviations 0.018 and 0.010: a 12.5 % gap at $z = -1.6$, outside the

module	comparison (same inputs)	max difference
transport	four real vortex tracks (800 samples), every estimate and error, non-default options	2×10^{-14} relative (two estimators bit-identical)
thermal	one shared thermal field ($N = 32$): norm, energy, T (two windows), condensate fraction, J_L , J_T , vortex count	$\leq 3 \times 10^{-14}$; count exact
thermal	the same field after 20 time units	2.7×10^{-12}
scattering	short case, three runs: positions, momentum, energy, slopes	7×10^{-14} , 8×10^{-13} , 3×10^{-14}
scattering	full probe ($L = 64$, $N = 128$, 400 time units): $\sigma_{\parallel}$, $\sigma_{\perp}$	identical to six digits (2.03913, 1.58133)
scattering	the registered matrix (31 runs, 15 points)	4.6×10^{-11} relative
vortex	imprint at positions with a vortex on a grid node; $T = 0$ separation at $t = 5, 40$ ($N = 64$)	10^{-4} (after a defect fix)
extension	run 20 time units ($N = 64$); <code>imprint_v2</code> ; <code>detect</code> ; <code>analyse_tracks</code>	10^{-10} ; 10^{-11} ; identical; bit-identical

Table 2: Rust against Python on identical inputs. The random-number generators differ (xoshiro256** against numpy’s PCG64), so constructions that draw random numbers (`random_state`, `heat`) are compared statistically, measurements on a given field exactly.

10 % criterion first set and not significant; a 60-seed comparison at another configuration cannot distinguish the two (vortex count 0.67 against 0.57, condensate fraction 0.739 ± 0.009 against 0.727 ± 0.014). The test asserts $|z| < 2.5$; we report the gap rather than the criterion.

Platform dependence. The scattering cross-check passes at 10^{-13} on x86-64 Linux, where its numpy fixtures were made, but its $T = 0$ control run differed by 8×10^{-3} in position on an Apple-silicon CI runner: a pair near the unstable saddle at $d = L/2$ amplifies rounding differences between libm and FFT kernels. The tolerances are strict on the reference platform and relaxed elsewhere.

4 What porting found

1. **A defect in the first Rust version**, found by the vortex cross-check: a vortex centred exactly on a grid node was imprinted with unit modulus instead of zero. Fixed; the separation now agrees to 10^{-4} .
2. **A bias in the Python programme’s friction estimator.** The port of the estimators was bit-identical to Python on real tracks; running the registered synthetic gate over many seeds then showed that the energy estimator of α is biased low by vortex diffusion: for a true $\alpha = 0.02$ it returns 0.0200 at $\eta = 0$ and 0.0183, 0.0163, 0.0156 at $\eta = 5 \times 10^{-4}$, 10^{-3} , 2×10^{-3} (−8, −18, −22 %), independently of the detection noise, while the regression estimator is unbiased (Fig. 1, right). The registered gate had passed on a single seed (its criterion passes on 2 of 12). The programme’s published friction values were corrected [4, 5].
3. **A hidden dependency of the solver library:** the Adams-order defect above, exposed by the first Hamiltonian system run through CVODE.

5 Performance

Method. The problem is identical in every engine: $N \times N$ grid, $L = N/2$, $g = 1$, $\Delta t = 0.01$, cutoff $k_{\max}/2$, one fixed initial state; the final states of all engines agree to $1e-13$ (relative difference of the checksum $\sum |\text{Re } c| + |\text{Im } c|$), so the timings are for the same computation.

Engines: numpy (pocketfft) [7], scipy.fft with one and four workers [8], torch on CPU with one and four threads [9], and **qf-pgpe** on one thread. Statistic: the best of the repetitions (three and five, in two measurement sessions) of the time per step, taken for every engine in the same way. **Machine and conditions:** Intel Core i7-4930MX (2013, 4 cores/8 threads, AVX), Linux, no GPU; during the measurement an unrelated four-thread training job of another project was running, so absolute times are pessimistic and the best-of statistic is used for all engines alike. JAX could not be imported in this environment, FFTW-based C codes and GPU codes were not available: **no comparison with them is made.**

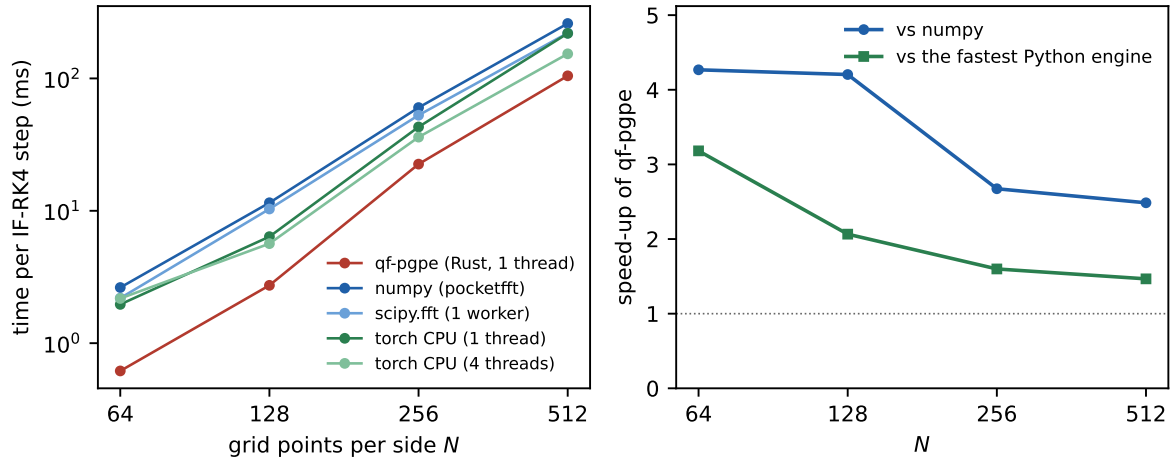


Figure 2: Time per IF-RK4 step against grid size for the engines of the benchmark (left) and the speed-up of **qf-pgpe** on one thread (right).

Results. On one thread **qf-pgpe** takes 0.62, 2.73, 22.52 and 104.60 ms per step at $N = 64, 128, 256, 512$, against 2.63, 11.50, 60.22 and 260.00 ms for numpy: speed-ups of 4.3, 4.2, 2.7 and 2.5. Against the fastest Python engine at each size (torch on CPU, which is multi-threaded in its four-thread form) the speed-up is 3.2, 2.1, 1.6 and 1.5. The advantage shrinks with N . We did not profile the origin of the advantage; the step is dominated by FFTs, and the optimisations we measured separately (allocation-free loop, unit-stride transforms, pruned transforms) are each of the order of ten to seventy per cent.

Throughput and end-to-end use. The programme’s experiments are many independent runs (seeds, pair sizes, wavenumbers), so throughput is the relevant quantity: with no interpreter lock and no per-step round trip, one process delivers 300 steps per second at $N = 128$ (numpy: 42) and the best aggregate over the concurrency we tried is 583 against 120, a ratio of 4.9 (Fig. 3). End to end: a 50-time-unit tracked $T = 0$ vortex run at 128^2 takes 41 s of CPU in Rust against 106 s in numpy (2.6 $\times$), and a thermal-state measurement (**run_blocks**, 60 time units with measurements, $N = 64$) takes 20.8 s with the drop-in extension against 109.8 s (5.3 $\times$) with the same T , n_s/n , vortex count and energy (the measurement code is the same Python).

6 The Python extension

The extension **qf_pgpe** (pyo3 and the numpy crate) exposes **Pgpe** (**run**, **step**, **invariants**, **imprint_v2**, **detect**, **momentum_band**) and **analyse_tracks** on C-contiguous **complex128** arrays of the numpy layout; **run** evolves for a given time inside one call with the interpreter lock released. A subclass of the programme’s Python stepper replaces only the stepping:

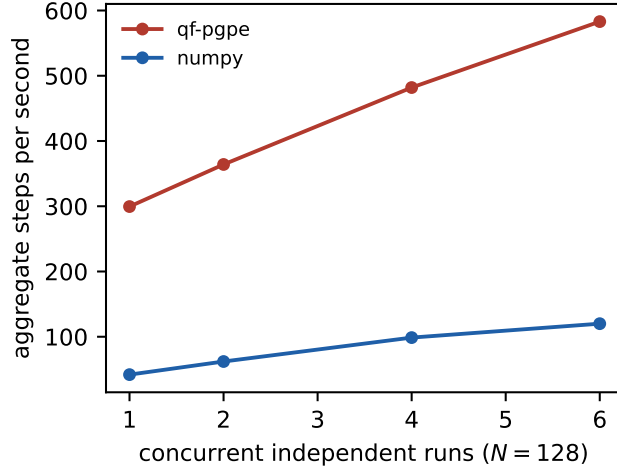


Figure 3: Aggregate throughput of independent runs ($N = 128$, one thread each) against the number of concurrent processes, on the same machine and under the same external load.

```
from pgpe_rust import PGPERust as PGPE      # every script that did 'from pgpe import PGPE' now steps in
Rust
s = PGPE(N=128, L=64.0); c = s.run(c, 100.0) # callbacks keep their contract (chunked evolution)
```

so the measurement code is unchanged. A smoke test (the exact plane wave, conservation, imprint and detection, input validation) accompanies the crate.

7 Use in the research programme

The tools have been used in a pre-registered programme whose results are published separately: the vortex-transport paper [4] (friction, diffusion and screening of vortices in a closed field), the friction-law paper [5] (the friction follows the temperature, not the normal density), and the software record [6]. Two uses are specific to this module. First, the 31-run vortex-phonon scattering matrix was reproduced independently in Rust, point by point, to 5×10^{-11} ; the follow-up with a larger box (amendment WS-A2) and the replication of the pair-separation matrix are run on it. Second, the estimator bias of Section 4 would not have been found by re-running the same code: it needed a second implementation and a seed sweep that was cheap enough to run.

8 Limitations and future work

- One machine, one CPU generation, a concurrent external load; no GPU and no comparison with other published solvers. A GPU backend and a comparison with FFTW-based C codes are the obvious next measurements.
- Intra-run parallelism did not help here (cause not investigated); large single simulations on many cores are not served.
- The thermal constructors use their own RNG; reproducing a numpy-seeded construction bit for bit is not possible and not a goal.
- The module is two-dimensional and periodic; the programme's three-dimensional and trapped geometries are not covered.
- Momentum is conserved to $\sim 10^{-6}$ (not 10^{-10}) for noisy fields with weight at the cutoff edge, where the cubic term aliases; the numpy engine shows the identical drift.

9 Availability

Source: <https://github.com/xaviercallens/rusty-SUNDIALS> (crates `qf-pgpe`, `qf-pgpe-py`; merged as pull request #69, commit 72cad84, 2026-10-09); BSD-3-Clause. The Python programme, the benchmark script (`exploration/pgpe/bench_engines.py`), the data (`data/generated/pgpe/bench`) and the ledger are in [6]. Reproduction: `cargo test -release -p qf-pgpe` (27 tests), `python3 exploration/pgpe/bench_engines.py single`.

References

- [1] M. J. Davis, S. A. Morgan, and K. Burnett. Simulations of Bose fields at finite temperature. *Phys. Rev. Lett.* *87*, 160402, 2001.
- [2] P. B. Blakie, A. S. Bradley, M. J. Davis, R. J. Ballagh, and C. W. Gardiner. Dynamics and statistical mechanics of ultra-cold Bose gases using c-field techniques. *Adv. Phys.* *57*, 363–455, 2008.
- [3] A. C. Hindmarsh, P. N. Brown, K. E. Grant, S. L. Lee, R. Serban, D. E. Shumaker, and C. S. Woodward. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM Trans. Math. Softw.* *31*, 363–396, 2005.
- [4] Xavier Callens. Vortex transport and screening in a closed two-dimensional bose field (version 2.1). Zenodo, 2026. doi:10.5281/zenodo.23225293.
- [5] Xavier Callens. The friction on a quantized vortex in a truncated gross-pitaevskii field follows the temperature, not the normal density. Zenodo, 2026. doi:10.5281/zenodo.23262132.
- [6] Xavier Callens. Socrateai-scientific-quantumfluids: Lean 4 library, pre-registrations and numerical record (v1.17.0). Zenodo, 2026. doi:10.5281/zenodo.23225129.
- [7] C. R. Harris et al. Array programming with NumPy. *Nature* *585*, 357–362, 2020.
- [8] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat. Methods* *17*, 261–272, 2020.
- [9] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* *32*, 2019.